

User Space und Kernel Space

Schutzgrenze, Übergänge und gemeinsame Schnittstellen verstehen

1.05

Die Grenze schützt Kernel und Prozesse voreinander

01 User Space

Anwendungen laufen mit begrenzten Rechten und in getrennten virtuellen Adressräumen.

02 Kernel Space

Kernelcode verwaltet Hardware, Speicher und globale Systemressourcen mit hohen Privilegien.

WARUM DIE TRENNUNG?

Fehler begrenzen

Zugriffe prüfen

Ressourcen koordinieren

Die Grenze ist eine technische Schutz- und Kontrollgrenze – keine organisatorische Konvention.

Ein Prozess sieht einen eigenen virtuellen Adressraum

User-Bereich

Programmcode · Heap · Stacks · Bibliotheken

Nicht gemappt / geschützt

Ungültige Zugriffe erzeugen einen Fault

Kernel-Abbildungen

Nur im privilegierten Modus zugreifbar

Virtuelle Adressen werden über Page Tables auf Speicher und Dateien abgebildet.

Ein Pointer ist nur im Kontext des zugehörigen Adressraums sinnvoll.

CPU-Modus und Page-Protection erzwingen die Grenze

01 Privilegstufe

Die CPU unterscheidet unprivilegierte Ausführung von privilegiertem Kernelcode.

02 Page Permissions

Page Tables begrenzen Lesen, Schreiben und Ausführen abhängig vom Modus.

VERSTOSS

User-Code greift auf geschützte Adresse zu

→ **Exception / Page Fault**

Die Hardware liefert die Mechanik; der Kernel definiert und verwaltet die Regeln.

System Calls sind der reguläre Eintritt in den Kernel

- | | | |
|----|----------------------------|--|
| 01 | Bibliotheksaufruf | z. B. <code>read()</code> oder <code>mmap()</code> |
| 02 | Syscall-Instruktion | CPU wechselt kontrolliert den Modus |
| 03 | Entry-Code | Register, Stack und Kontextzustand sichern |
| 04 | Kernel-Funktion | Argumente prüfen und Subsystem aufrufen |
| 05 | Return | Ergebnis, Fehler und offene Arbeit behandeln |

Ein System Call ist ein definierter Zustandswechsel – kein direkter Sprung zu Hardware.

Die ABI legt Nummern, Register und Rückgabewerte fest

01 Syscall-Nummer

Die Architektur ordnet dem angeforderten Dienst eine Nummer und einen Entry-Pfad zu.

02 Argumente

Einfache Werte werden gemäß ABI über Register oder Stack übergeben.

VEREINFACHTE SICHT

```
read(fd, buffer, count)
```

```
return >= 0  Anzahl Bytes
```

```
return < 0   Fehler via errno
```

Die Userspace-ABI muss stabil bleiben; interne Kernel-Schnittstellen dürfen sich verändern.

User-Pointer werden nie blind dereferenziert

01 Adresse prüfen

Ein vom Prozess gelieferter Pointer kann ungültig sein oder auf nicht zugänglichen Speicher zeigen.

02 Daten sicher kopieren

Kernel-Helfer übertragen Daten zwischen User- und Kerneladressraum und behandeln Faults.

DATENRICHTUNG

User → Kernel

`copy_from_user()`

Kernel → User

`copy_to_user()`

Die Prozessgrenze bleibt auch während eines System Calls relevant.

Kernelcode läuft in unterschiedlichen Kontexten

Prozesskontext

Kernel arbeitet im Auftrag eines Tasks; Schlafen kann erlaubt sein

Kernel-Thread

Task ohne normalen User-Adressraum; führt Kernelarbeit aus

Interruptkontext

Asynchron; keine normale Prozessausführung und starke Einschränkungen

Deferred Work

Arbeit wird in einen geeigneteren späteren Kontext verlagert

Exceptions, Signale und Fehler sind verschieden

01 Exception

Die CPU meldet ein synchrones Ereignis wie Page Fault oder ungültige Instruktion.

03 Fehlercode

Ein System Call liefert einen negativen Kernel-Fehler, den die Bibliothek als errno sichtbar macht.

02 Signal

Der Kernel informiert einen Prozess über ein Ereignis; der Prozess kann reagieren oder beendet werden.

04 Kein Automatismus

Nicht jede Exception beendet einen Prozess: Ein Page Fault kann eine gültige Seite nachladen.

Entscheidend ist, ob der Kernel das Ereignis behandeln kann oder an den Prozess weitergibt.

mmap() und Page Faults verbinden beide Welten

- | | | |
|----|-------------------|---|
| 01 | mmap() | Prozess fordert eine virtuelle Abbildung an |
| 02 | VMA | Kernel beschreibt Bereich, Rechte und Quelle |
| 03 | Zugriff | CPU entdeckt fehlende oder geschützte Abbildung |
| 04 | Page Fault | Kernel lädt, erzeugt oder verweigert die Seite |
| 05 | Weiterlauf | Instruktion wird wiederholt oder Signal ausgelöst |

Virtueller Speicher macht Datenzugriff bedarfsgesteuert statt vollständig vorab.

Dateideskriptoren kapseln Kernelobjekte

01 Handle im User Space

Der Prozess erhält eine kleine Zahl statt eines direkten Zeigers auf ein Kernelobjekt.

02 Objekt im Kernel

Die Descriptor-Tabelle verweist auf ein File-Objekt mit Operationen und Zustand.

EINHEITLICHE OPERATIONEN

`read()`

`write()`

`poll()`

`close()`

Dateideskriptoren vereinheitlichen Dateien, Pipes, Sockets und viele Geräte.

Kernel-Schnittstellen sind mehr als System Calls

Dateien und Geräte

read · write · ioctl · poll

procfs

Prozess- und Laufzeitinformationen

sysfs

Kernelobjekte, Geräte und Attribute

Netlink und weitere APIs

Strukturierter Nachrichtenaustausch für bestimmte Subsysteme

Sicherheit wird an mehreren Stellen geprüft

01 Credentials

UID, GID und Prozess-Credentials bilden die Identität für Zugriffsentscheidungen.

03 LSM und Richtlinien

Zusätzliche Sicherheitsmodule ergänzen klassische Prüfungen an definierten Hooks.

02 Capabilities

Privilegierte Fähigkeiten werden feiner als ein einziges Root-Recht aufgeteilt.

04 seccomp

Ein Prozess kann die Menge nutzbarer System Calls begrenzen.

Ein erfolgreicher Moduswechsel bedeutet noch nicht, dass die angeforderte Operation erlaubt ist.

Praxis: Die Grenze lässt sich beobachten

USER- UND KERNEL-SICHT

```
$ strace -e openat,read,write cat  
/proc/version  
$ cat /proc/<PID>/maps  
$ cat /proc/<PID>/status  
$ ls -l /proc/<PID>/fd  
$ dmesg --level=err,warn
```

01 strace

Zeigt System Calls, Argumente, Rückgabewerte und Fehler eines Prozesses.

02 /proc

Zeigt Adressraum, Status und File Descriptors aus Kernelperspektive.

Werkzeuge beobachten Schnittstellen und Zustände – sie heben die Schutzgrenze nicht auf.

Beispiel: read() überquert die Grenze zweimal

- | | | |
|----|---------------------|---|
| 01 | User Space | Buffer und Länge vorbereiten |
| 02 | Entry | Syscall-Zustand herstellen |
| 03 | Kernel Space | FD prüfen · VFS aufrufen · Daten beschaffen |
| 04 | Kopieren | Daten sicher in den User-Buffer übertragen |
| 05 | Return | Anzahl Bytes oder Fehler an Prozess zurückgeben |

Drei Leitfragen: Wer ruft? Welche Daten queren die Grenze? Welche Prüfung entscheidet?